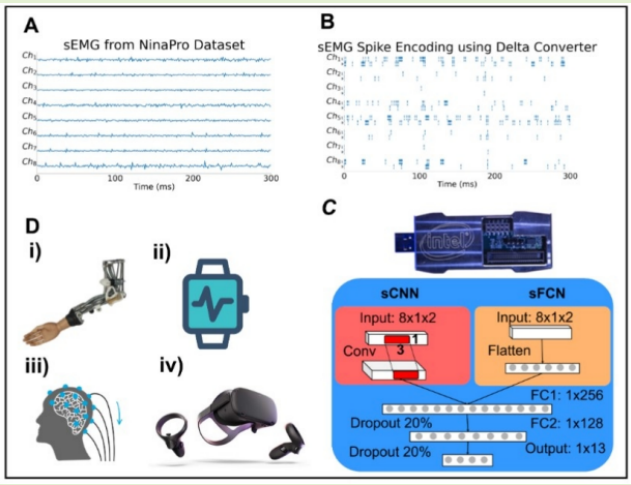
Neuromorphic Edge Computing for Biomedical Applications: Gesture Classification Using EMG Signals

Antonio Vitale, Elisa Donati, *Member, IEEE*, Roger Germann, and Michele Magno, *Senior Member, IEEE*

***Abstract*—With the emergence of edge-computing platforms, the applications of smart wearable devices are immense. This technology can be incorporated in consumer products such as smartwatches and activity trackers, for continuous health monitoring, as well as for medical applications such as myoelectric prosthetics, to interpret the electric activity in the residual limb and achieve fast and precise control. However, wearable technologies require a lightweight, energy-efficient, and low-latency processing system in order to extend the devices' autonomy while maintaining a realistic user-feedback interaction. Neuromorphic processing, thanks to its event-based and asynchronous nature, presents ideal characteristics for compact brain-inspired low-power and ultra-fast computing systems that can enable a new generation of wearable devices. This article presents two spiking neural networks (SNNs) for event-based electromyography (EMG) gesture recognition and their evaluation on Intel's research neuromorphic chip Loihi. Specifically, the evaluation is done on the Kapoho Bay platform which embeds the Loihi processor in a Universal Serial Bus (USB) form factor device allowing for closed-loop edge computation. With accurate experimental evaluation, this article demonstrates that the proposed method is able to discriminate 12 different hand gestures using an eight-channel EMG sensor and exceeds state-of-the-art results. We obtained an accuracy of 74% on the commonly used NinaPro DB5 dataset, a processing latency of 5.7 ms for 300-ms EMG samples while consuming only 41 mW.**



***Index Terms*— Deep learning, edge-computing, electromyography (EMG) signal processing, event-based programming, neuromorphic hardware, spiking neural networks (SNNs).**

I. Introduction

A WEARABLE biomedical device is an autonomous, non-invasive system that interacts with the human body to perform a specific medical function such as vital signs monitoring or control of prosthetics [1]. These devices can also send this information to doctors or other healthcare professionals in real-time [2], [3]. Among different applications like sport tracking [4], personalized health monitoring [5], virtual and augmented reality [6], and several biomedical applications [7], wearable technology is very promising to design a new generation of prosthetic devices to improve their performances making them more compatible for everyday activities [8]. They can be used to read the activity of the residual muscle and therefore either control a prosthetic device [9], improve assistive rehabilitation robots [10], or bring additional functions such as the sensation of touch and pain [11]. The design of wearable technology requires energy efficiency, to extend the autonomy of such battery-powered devices, and fast processing times, to provide more realistic user feedback. Therefore, resource-constrained devices are crucial building blocks for their implementation [12].

There has been an increasing interest in the development of circuits and systems to enable edge-computing for wearable technologies [18]. Edge-computing allows continuous signal

Manuscript received 31 March 2022; revised 1 July 2022; accepted 19 July 2022. Date of publication 3 August 2022; date of current version 14 October 2022. This work was supported by the European Union's ERA-NET CHIST-ERA 2018 Research and Innovation Programme under Grant ANR-19-CHR3-0008. The associate editor coordinating the review of this article and approving it for publication was Dr. Edward Sazonov. *(Corresponding author: Antonio Vitale.)*

Antonio Vitale, Roger Germann, and Michele Magno are with the Center for Project-Based Learning, ETH Zürich, 8092 Zürich, Switzerland (e-mail: antonio.vitale@pbl.ee.ethz.ch; roger@germann.uk; michele.magno@pbl.ee.ethz.ch).

Elisa Donati is with the Institute of Neuroinformatics, University of Zurich and ETH Zürich, 8057 Zürich, Switzerland (e-mail: elisa@ini.uzh.ch).

Digital Object Identifier 10.1109/JSEN.2022.3194678

1558-1748 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

monitoring with decreased power consumption and lower computation times since signals are processed directly at the sensor side. Furthermore, by avoiding the transmission of sensitive information to external processing systems, the data remain on the device, thus avoiding privacy-related issues [19]. This real-time monitoring and processing of physiological data enable a faster, more energy-efficient closed-loop interaction with the environment [20].

Neuromorphic processors represent a promising approach toward the development of energy-efficient and low-latency edge-computing wearable devices [21]. These processors mimic the natural biological structures found in the nervous system by implementing neurons and synapses directly in silicon and they are able to process information in a massively parallel fashion while consuming very low power [22]. Their electronic circuits are typically designed using digital [23]–[25] or mixed-mode analog/digital transistors [26] and are fabricated using standard very large-scale integration (VLSI) processes to emulate the physics of real neurons and synapses in real-time [27].

Neuromorphic devices implement brain-inspired computational models of standard artificial neural networks (ANNs) called spiking neural networks (SNNs) directly in hardware. The computational elements of SNNs are biologically plausible spiking neurons and represent a natural processing framework for event-based sensors which can detect light [28], [29], sound [30], and even chemical [31] signals. Unlike classical ANNs, due to SNNs' explicit time dependency in their computations, they are able to detect information encoded in spatio-temporal patterns of spiking signals [32]. spiking neural networks (SNNs), in fact, enable encoding and processing of neural information through precisely timed spike trains [33] and they have shown promising results in the processing of biomedical signals such as electromyography (EMG) [34], [35], electrocardiogram (ECG) [36], [37], or high frequency oscillation (HFO) [38].

The main trends on EMG data processing for classification tasks use either traditional machine learning (ML) algorithms, such as support vector machines (SVMs), or more complex deep ML methods [39]. Conventional methods rely on extracting certain features in order to classify the raw signals. Feature extraction techniques, relevant to electromyography (EMG) applications, can be categorized in *time domain*, for example, root mean square (rms) and mean absolute value (MAV), *frequency domain*, Fourier transform, and *time–frequency domain*, short-time Fourier transform (STFT) and marginal discrete wavelet transform (MDWT), and continuous wavelet transform (CWT) [40]. The work in [41] implements an SVM to process EMG signals by first extracting five commonly used signal features and then classifying the corresponding gestures. Chen *et al.* [42] first transform the EMG signals using a CWT, then train a four-layer convolutional neural network (CNN). The approach in [43] applies transfer learning (TL) to data from multiple users to increase prediction accuracy. After transforming the data in the same way using a CWT, they are able to achieve higher accuracies. Wei *et al.* [44] achieved high accuracy by extracting different time- and frequency-domain features before running the classification with a multiview CNN. These methods require preprocessing of the EMG data before running the computationally expensive models, thus not being ideal for wearable applications. None of them report power consumption or inference time. Zhang *et al.* [45] run an EMG classification with an Artificial Neural Network (ANN) using a custom, five-gesture dataset and obtain an inference time of 227.6 ms. In contrast to these works, our proposed method operates on raw, spike-encoded EMG data and uses a lightweight SNN running on a low-power edge device. We achieve an accuracy of 74% on a popular dataset with an inference time of 5.7 ms while consuming only 41 mW. Table I summarizes methods used for EMG signal processing found in literature. We report the task description and methodology and, where available, the platform on which the classification algorithm runs as well as the achieved accuracy, inference speed, and power consumption.

Recently, promising methods exploit SNNs running on neuromorphic processors for EMG-based gesture classification. In [46], an SNN is designed to map, learn, and understand spatio-temporal brain data and reveals a better classification accuracy compared to traditional ML for estimation of finger forces from four different hand poses. In [17], different SNN networks were compared for a hand-gesture classification task, where the EMG signal was fused with a spiking visual sensor. The comparison was performed in terms of accuracy, power consumption, and latency showing that the network is able to obtain the same accuracy with a significantly smaller energy-delay product (up to 600× more efficient) than an equivalent network running on a graphical processing unit (GPU). Although traditional ML methods achieve good results in terms of accuracy, neuromorphic computing shows orders of magnitude gains in power and performance over conventional digital architectures, such as a central processing unit (CPU) and GPU, thanks to their sparse, event-driven processing.

The primary goal of the work presented in this article is to assess the feasibility of running an EMG-based classification algorithm directly on a digital neuromorphic edge device, to minimize the power consumption and latency while achieving state-of-the-art classification accuracy. The final goal is to exploit the low-latency output of this classification scheme for closed-loop applications, such as the design of an energy-efficient system for prosthesis control. The main contributions of the article are summarized as follows.

1) *The design and implementation of two spiking networks—a spiking CNN (sCNN) and a spiking fully connected network (sFCN)—on a neuromorphic processor to classify event-based EMG data.*
2) *The evaluation and the comparison of these in terms of accuracy, energy efficiency, and latency.* The two SNNs were implemented on Intel's digital neuromorphic chip Loihi. Specifically, the SNNs were implemented on the Kapoho Bay platform, an embedded Loihi system as a Universal Serial Bus (USB) form factor that enables edge-computing. The accuracies of the SNNs were compared to classification results obtained using a more traditional ML, namely an SVM. This work uses the open NinaPro DB5 dataset [47], where the raw EMG signals are encoded into spike trains

TABLE I
SUMMARIZING DIFFERENT METHODS FOUND IN LITERATURE TO PROCESS EMG AND EEG SIGNALS. FOR EACH PAPER, WHERE AVAILABLE, WE REPORT THE PROCESSING PLATFORM, THE ACHIEVED ACCURACY, THE INFERENCE SPEED, AND THE ENERGY/POWER CONSUMPTION

References	Task	Approach	Device	Dataset	Accuracy	Speed	Consumption
Zanghieri et al. 2020 [13]	Hand gesture classification running on custom computing platform	TCN architecture to classify 150 ms raw sEMG signals	GAP8 embedded 8-core processor	NinaPro DB6	69.2%	12.82 ms	0.9 mJ
Tsinganos et al. 2019 [14]	Hand gesture classification	TCN for time sequence classification using raw EMG signals	Nvidia GTX1080	NinaPro DB6	89%	N/A	N/A
Tam et al. 2019 [15]	Hand gesture classification using custom 32-channel HD-sEMG sensor	2D CNN and "frequency-time-space cross-domain" preprocessing	Nvidia Jetson Nano	Custom 6-gesture dataset	98%	100-200 ms	49 mW(sensor) + 3.05 W(processor)
Cisnal et al. 2021 [16]	Assist robotic exoskeleton using embedded EMG-based control	Custom 2-channel EMG sensor and processing on a MCU	MCU: TMS320F28069M	Custom 3-gesture dataset	97%	0.48 s	N/A
Wang et al. 2020 [12]	Embedded motor-imagery Brain Computer Interface	CNN based on EEGNet architecture	MCU: Cortex-M4F	Physionet EEG Motor Movement/Imagery dataset	82.4%	101 ms	4.28 mJ
Ceolini et al. 2021 [17]	Fully neuromorphic sensor fusion approach for hand-gesture classification	SNN running on neuromorphic chip to classify spike-encoded 200 ms EMG+DVS signal	Intel's Loihi[1] and ODIN+MorphIC[2]	Custom 5-gesture EMG-DVS dataset	$96\%^1$ and $89.4\%^2$	$7.75ms^1$ and $19.5ms^2$	$1104uJ^1$ and $37.5uJ^2$
This work	Hand gesture classification using 300 ms raw EMG signals	SNN running on a Neuromorphic Chip	Intel's Kapoho Bay	NinaPro DB5	74%	5.7 ms	41 mW

using a delta-encoding converter algorithm that emulates the behavior of its hardware implementation described in [38]. The SNNs were directly trained in the spiking domain, using the open PyTorch Spike Layer Error Reassignment (SLAYER) implementation [48] and then deployed to the Loihi to obtain accuracy, power, and latency and measurements.

3) *The experimental evaluation of the effectiveness of the proposed solutions.* This article shows that the achieved accuracy with the conventional SVM remained below 70%, while the two proposed SNNs achieved higher, state-of-the-art accuracy. Moreover, we demonstrate the energy efficiency by comparing the latency and power consumption of an equivalent classification scheme running on a GPU. Fig. 1 shows a high-level schematic of the whole processing pipeline.

II. MATERIALS AND METHODS

A. Dataset

NinaPro datasets [47] are commonly used in gesture classification literature [42], [49], [50]. In this article, we used the DB5 EMG dataset. The dataset features ten non-disabled subjects (eight males and two females) and uses two Myo bands to measure the EMG data. The bands have a sampling rate of 200 Hz, a resolution of 8 bits, and eight sensor channels. This work used the data from one Myo Band and a subset of the dataset, Exercise A, where the subjects alternate between performing one of the possible 12 gestures for 5 s and returning to the rest position for 3 s. The NinaPro DB5 dataset is particularly challenging because of the following.

1) It is highly unbalanced, with almost 60% representing the resting position (zero label) due to the fact that the subjects return to the rest position between every gesture.
2) It has a relatively small amount of data per gesture and a relatively large number of different gestures.
3) The gestures are highly similar. The spectral decomposition of the EMG signals for two different gestures is almost identical [42].

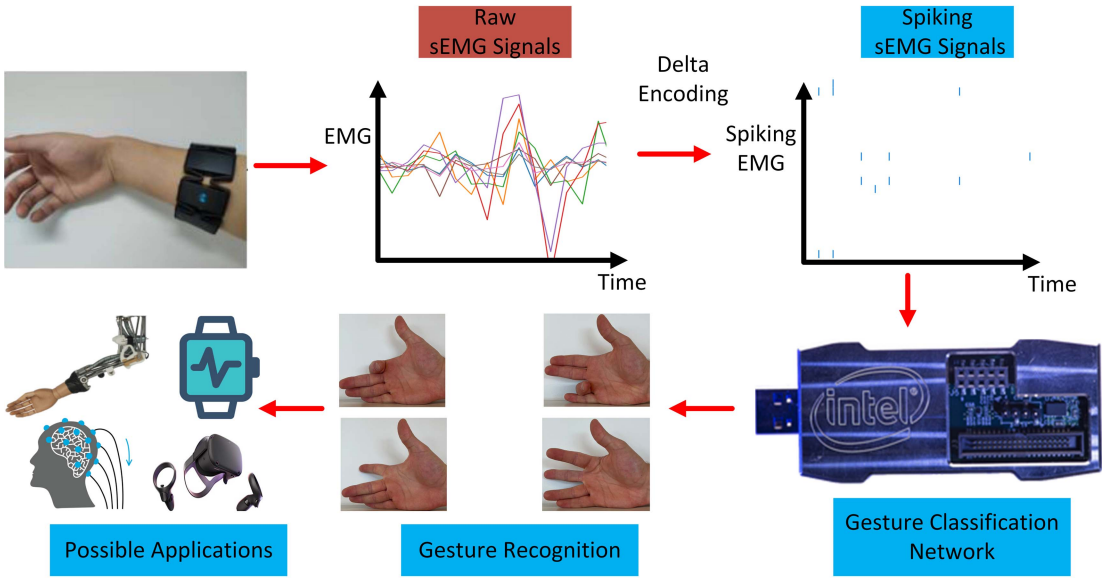


Fig. 1. Schematic of the proposed EMG classification system. The raw sEMG signals are converted to equivalent spiking signals before being sent to the Neuromorphic Hardware. The classification network running on the device outputs the detected gesture. Possible applications include prosthesis control, health and vital signs monitoring, brain–machine interfaces (BMIs), and movement prediction for virtual reality (VR).

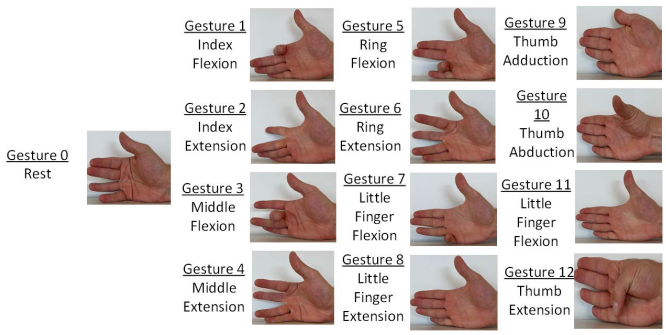


Fig. 2. Replicated images of the gestures used in the NinaPro-DB5 dataset.

The first step in the classification procedure is separating the recorded signal with their corresponding label into samples of 300 ms, a common value used in literature [45], [51]. To increase the number of available sample points, we upsampled the signal. The signals were interpolated using linear interpolation. For a signal of length N and using an interpolation factor K, the resulting new signal has a length of $(N-1)*K+1$. Here, an interpolation factor $K=5$ was chosen, that is, the sampling frequency was increased from 200 to 1000 Hz. The data were split following the NinaPro standard [51]: the experiments were repeated a total of six times per subject and repetitions 2 and 5 are used in the testing dataset, while the remaining repetitions are included in the training dataset. Fig 2 shows the gestures as well as their associated classification labels.

B. From Time to Spike Domain: Signal Encoding

Traditional sensors, such as EMG sensors, are not able to output their information in a spiking manner: the outgoing signals have to be converted to equivalent spiking signals before being processed by SNNs [52].

The most common approach is the delta converter circuit [38] functionally equivalent to the one used in the dynamic vision sensor (DVS) [28]. Our adapted circuit produces two asynchronous digital pulse outputs (UP or DOWN) for every EMG input channel. The UP (DOWN) spikes are generated every time the input signal exceeds a specific positive (negative) threshold ϵ. This approach was used to convert EMG signals used in mixed-signal neuromorphic chips [35], [53] and in digital ones [17], [46], as well as electrocardiogram (ECG) [36], [37] and electroencephalogram (EEG) signals [38], [54]. This method was adapted for the fully-digital Loihi by assigning a *spike polarity* of 1 or 0 for an UP or a DOWN spike, respectively. Fig. 3 illustrates this spike encoding mechanism for a random EMG sample.

The relationship between the continuous surface electromyography (sEMG) signal $s(t)$ and its spiking counterpart $\hat{s}(t)=\{x,y,p,\hat{t}\}$ is given by the following equation:

$$\begin{aligned} x &\in 0,7 \\ y &= 0 \\ p &= \begin{cases} 1, & s(t)-s(t+1)>\epsilon \\ 0, & s(t)-s(t+1)<-\epsilon \end{cases} \\ \hat{t} &= n, |s(n*T_{\text{sample}})-s(n*T_{\text{sample}}+1)|>\epsilon \end{aligned} \tag{1}$$

where x is the active EMG channel ranging between zero and seven, p is the spike polarity corresponding to an UP/DOWN spike, while $\hat{t}$ corresponds to the Loihi timestep at which the spike is injected in the network. Using a sampling time of $T_{\text{sample}}=1$ ms and a sample duration $T_{\text{duration}}=300$ ms, $\hat{t}$ ranges between 0 and 300. A threshold of $\epsilon=3$ was used.

C. Conventional ML: SVMs

SVMss are supervised learning models that analyze data for classification and regression tasks by finding hyperplanes within the dataset, which separate samples in their respective

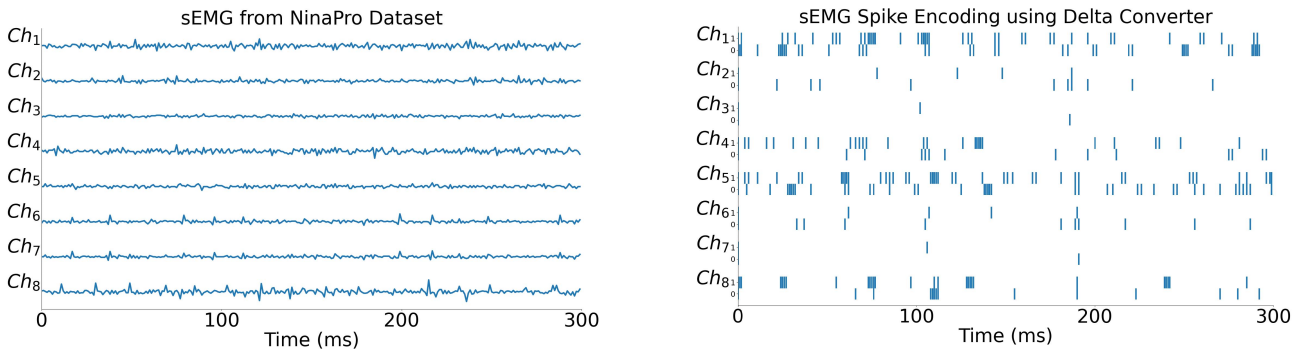


Fig. 3. **Left:** raw EMG signal. **Right:** encoded EMG signal using the adapted delta converter [38]. For every EMG input channel, the equivalent spiking signal has two possible outputs (UP or DOWN). The UP (DOWN) spikes are generated every time the input signal exceeds a positive (negative) threshold. In our encoding, an UP spike corresponds to a spike polarity of 1, a DOWN spike corresponds to a spike polarity of 0.

categories [55]. In this work, we used both a linear and nonlinear SVM as a baseline to quantitatively compare the performance of conventional ML approaches to our SNN-based method on the same dataset. Using an SVM with either a linear or a radial basis function (RBF) kernel resulted in accuracy on the testing set of 64.1% and 67.7%, respectively.

Like in [17], we extracted both the rms and MAV values from the raw EMG samples. These two feature vectors were concatenated together to form the full feature vector used for the classification task. A hyperparameter sweep showed that for EMG samples of 300 ms, a window size of 50 ms resulted in the best classification accuracies.

As mentioned in Section II-A, the dataset used in this work is highly unbalanced. In order to penalize the classes with high sample counts, the SVM was weighted with a class-specific factor $C = (\kappa/n_{\text{samples}})$, where $\kappa = 1e5$ was empirically chosen to achieve best accuracy and n_{samples} is the number of samples for a given class.

D. Neuromorphic Processor-Intel's Loihi

In this work, Intel's neuromorphic research chip Loihi [56] was used to run the SNNs. The proposed designed algorithms were evaluated both on the Nahuku32 platform, running on Intel's cloud server, as well as on the Kapoho Bay platform. While both platforms utilize the same Loihi chip for computation, the Nahuku32 board consists of 32 Loihi chips and allows run-time performance characterization by using the *EnergyProbe* and *ExecutionTimeProbe* object directly from Intel's Application Programming Interface (API). The Kapoho Bay, as shown in Fig. 4, comes in a USB form factor with two Loihi chips containing a total of 256 neuro-cores where 262 144 neurons and up to 260 million synapses can be implemented. It can be easily interfaced for live communication with the host system (an UP board featuring an Intel[1] ATOM x5-Z8350 Processor with 64 bits up to 1.92 GHz and 4 GB of RAM. The UP board is running Ubuntu 18.04. Python 3.7 and version 1.0.0 of Intel's NxSDK are used to program the SNNs) by programming the three embedded ×86 processors which are used for monitoring and I/O spike management embedded on the Loihi chip.

[1]Registered trademark.

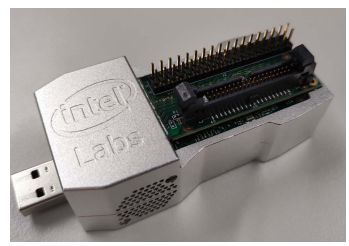


Fig. 4. Intel's Kapoho Bay.

E. Deep SNNs

1) Overview of SNNs: The neural model implemented on Loihi, given by (2) and (3), is a variation of the CUBA leaky-integrate-and-fire model [24]. Here, a spiking neuron has two state variables both with their own temporal behavior: the synaptic response current $i(t)$ and the membrane potential $u(t)$, which evolve according to the associated decay constants τ_i and τ_u. A spike train is mathematically modeled as a sum of Dirac delta functions $\sigma(t) = \sum_k \delta(t - t_k)$, where t_k is the spike time of the kth spike. This is then summed and filtered in its current $i(t)$ according to the following equation:

$$i(t) = \sum_{i \neq j} w_{ij}(\alpha_{ij} * \sigma_j)(t) + b_i \tag{2}$$

where w_{ij} corresponds to the weight connecting neuron j to neuron i, $\alpha_{ij}(t) = \tau_i^{-1} e^{(-t/\tau_i)} H(t)$ is the synaptic filter determined by the current decay constant τ_i, $H(t)$ is the unit step function, and b_i is a constant bias. The synaptic current is then further integrated into its membrane potential according to the following equation:

$$\dot{u}(t) = -\frac{1}{\tau_u} u(t) + i(t) - \Theta\sigma(t) \tag{3}$$

where Θ is the neuron's potential threshold. The time constant τ_u determines the leaky integrating behavior of the neuron. If a neuron's potential exceeds its threshold Θ, a spike is generated and propagated to postsynaptic neurons while its membrane potential $u(t)$ resets back to its resting potential $u(t)_{\text{rest}} = 0$.

2) SLAYER in Time: In contrast to ANNs, the activation function of spiking neurons is binary and thus nondifferentiable therefore training cannot be done using conventional gradient-based back-propagation. Techniques to train deep SNNs can be classified into two categories: *online* and *offline* approaches. Online approaches deploy an SNN to neuromorphic hardware and then use on-chip plasticity features to evolve the network parameters in real-time. Such methods include spike timing dependent plasticity (STDP) rules. On the other hand, offline approaches use a prerecorded dataset to train a model on a CPU or GPU and later implement the trained model on neuromorphic hardware. Tools such as NengoDL [57] or the SNN Conversion Toolbox [58] allow training of a standard ANNs and subsequent conversion to equivalent SNNs while other training frameworks, such as SpikeProp [59], EvSpikeProp' [60], and SLAYER [48], directly train SNN by approximating back-propagation in the spike domain.

In this work, the SLAYER framework is used to train and evaluate the different SNN models [48]. SLAYER differs from other direct SNN training methods, as it distributes the effect of input spikes into future time values by using a *spike response kernel* to solve the temporal error credit assignment problem and it approximates the derivative of a spike function as a *probability density function*. For classification tasks, a decision is made based on the number of output spikes during an interval rather than the precise timing of the spikes. The error function $e(t)$ for the output neuron n_l during this interval of length T_{int} is given by

$$e^{(n_l)}(t) = \int_{T_{\text{int}}} s^{n_l}(\tau)d\tau - \int_{T_{\text{int}}} \hat{s}^{n_l}(\tau)d\tau \tag{4}$$

where $s^{n_l}(t)$ is the output spike train of neuron n_l, $\hat{s}^{n_l}(t)$ is the number of target spikes during the interval, and **not** a target spike train itself.

III. Results

Section III presents the results of the energy-efficient SNN architectures used for gesture recognition. Section III-A focuses on the training of the networks and shows the effect of different network configurations on the resulting classification accuracy. Section III-B describes their implementation on the neuromorphic chip while benchmarking the run-time performance in terms of execution time, power and energy consumption as well as hardware usage. Section III-C compares the performance profile of the SNN running both on the Loihi and on a standard GPU. Finally, Section III-D reports the obtained accuracies and compares them to other SoA methods. The gesture-wise accuracies on the NinaPro DB5 testing set are also presented. This section may be divided into subheadings. It should provide a concise and precise description of the experimental results, their interpretation, as well as the experimental conclusions that can be drawn.

A. Implementation of SNNs on SLAYER

In this work, two fundamentally different architectures were tested and compared against each other: an sFCN and an sCNN. Both the spiking fully connected layers and spiking convolutional layers operate in the same way as those found in conventional ANNs. A fully connected layer consists of two spiking neuron populations connected in an all-to-all fashion while a spiking convolutional layer encodes the respective filter entries as synaptic weights. Both networks are feed-forward networks and were trained directly using the PyTorch SLAYER implementation running on a Computer Unified Device Architecture (CUDA)-enabled GPU.

The input to the networks are the spike-encoded EMG signals as described in Section II-A and are given by (1). To train SNNs using SLAYER, the input has to be 4-D: $x, y, p, \hat{t}$. Here, the x-dimension corresponds to the sensors ranging from 0 to 7, the y-dimension is kept at 0, a polarity of 1 or 0 is assigned for an UP or a DOWN spike, respectively, and the spike time $\hat{t}$ corresponds to the time at which an UP or DOWN spike is recorded. The input layers for all SNNs have dimensions $\{H, W, C\} = \{8, 1, 2\}$, where $H = 8$ corresponds to the number of sensors and $C = 2$ since each spike is an UP or a DOWN spike.

The same training and neuron parameters are used for both the sCNN and sFCN. The training procedure is run for a total of 100 epochs using a batch size of 10. The ADAM optimizer with a learning rate of 0.01 is used to minimize the error function given in (4), where the target spike count $\hat{s} = 200$ and time interval $T_{\text{int}} = [10, 200]$. All neurons have the same internal parameters. In this work, the membrane threshold $v_{\text{Threshold}} = 200$, the voltage decay $\tau_{\text{Voltage}} = 128$, and current decay $\tau_{\text{Current}} = 1024$.

As in conventional deep neural networks (DNNs), obtaining good accuracy in deep SNN depends on the network complexity. Models with low complexities result in them not being sufficiently generalizable, while using high complexities may result in overfitting the training data. Different hyperparameters, such as layer depth and layer size, determine the overall complexity. To find an optimal model structure, different sFCN and sCNN networks, each with different hyperparameter sets, were trained for 100 epochs and compared based on their respective obtained accuracies on the testing set. Note that all networks achieve a baseline accuracy of roughly 60% due to the fact that the data is unbalanced as mentioned in Section II-A. After running this parameter search, the best sFCN and sCNN are compared in terms of obtained accuracy.

Since both proposed SNN models require at least one fully connected layer, the sFCN was optimized first. Two network structures were compared using either one or two hidden fully connected layers. Fig. 5 shows the obtained accuracies for different networks where FC_1 and FC_2, respectively, denote the dimension of the first and second hidden fully connected layer. The plot shows that using two hidden fully connected layers results in better classification accuracy.

For the sCNN, the number of convolutional filters is an additional network hyperparameter. Also, here, two different network structures were tested with one or two hidden fully connected layers with varying numbers of filters. Fig. 6 shows the obtained accuracies on the testing set for different network

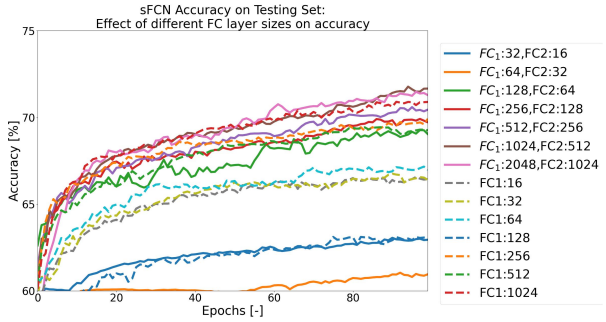


Fig. 5. Effect of different sFCN networks on classification accuracy of the testing set.

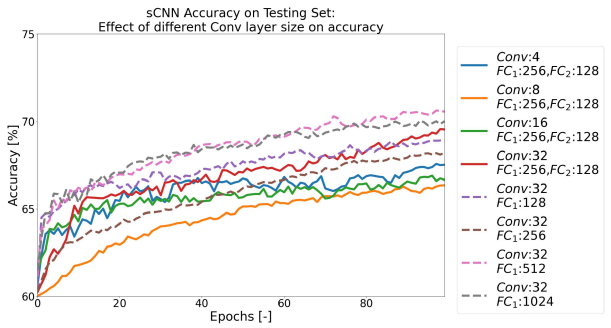


Fig. 6. Effect of different sCNN networks on classification accuracy of the testing set.

structures, where Conv corresponds to the number of filters, while FC_1 and FC_2, respectively, denote the dimension of the first and second hidden fully connected layer.

As shown in the curves, the sFCN outperforms the sCNN for all cases. This is in line with expectations since EMG signals encode information temporally and not spatially. Only the sFCN is further discussed in this work. Section III-B focuses on the implementation of the trained SNN on the Loihi Chip.

B. Implementation of SNNs on Loihi

The proposed networks are designed to have low latency and high energy efficiency; moreover, the low-dimensionality and sparse nature of the input spiking sEMG signal help to reduce the computational time. Important benchmarks to characterize the performance of the SNN are: the time needed to process a sample—the processing *latency* L_{sample}, the consumed *power* per sample P_{sample}, and the consumed *energy* per sample E_{sample}. The *latency* and *power* values can be obtained directly from Intel's API using the *ExecutionTimeProbe* and *EnergyProbe* for each algorithmic timestep, while the *energy* can be obtained by simply multiplying these two values together. From this energy probe, one can also separate the power consumed only by the Neurocores and the embedded ×86 CPUs, denoted as $P_{\text{neurocore}}$, from the power consumed by the whole system. Since P_{sample} considers all components in the processing pipeline, like host-side communication and spike monitoring, while $P_{\text{neurocore}}$ considers only the power consumed from the operations happening on the Neuromorphic Chip, the measured P_{sample} will be of orders of magnitude larger than $P_{\text{neurocore}}$.

The dataset is separated into samples of 300 ms and each SNN simulation timestep processes 1 ms of the input signal. An additional 300 *blank timesteps* are used after every sample in order to reset the neurons to their resting state. A total of 600 Loihi timesteps are needed to fully process a 300 ms sample.

Another important consideration when deploying an SNN on neuromorphic hardware is the physical allocation of the neurons on the neurocores. Hardware constraints, such as the maximum number of neurons per core as well as the maximum number of connections between cores, have to be respected when programming networks on any neuromorphic chip. Allocating too many highly interconnected neurons on the same neurocores results in communication congestion and thus longer processing latencies and higher energy consumption. On the other hand, distributing the neurons too sparsely in the neuromorphic chip results in high compilation times and a greater hardware occupation. To illustrate the effect of using different network sizes with a different *neuron-per-core* allocation, two networks—denoted as *small* and *large* networks—are compared in terms of latency L_{sample}, energy E_{sample}, and power P_{sample} & $P_{\text{neurocore}}$.

The inference runs on the neuromorphic hardware for a total of 1000 input EMG samples. The execution time and power measurements are only probed on the last 100 samples to ignore any transient effects in the hardware. These obtained values are then divided by 100 in order to obtain the measurements per sample. The experiments were repeated five different times and the averaged values were compared in terms of latency L_{sample} per sample in ms, the power P_{sample} consumption per sample in mW, the energy consumption E_{sample} per sample in mJ, as well as the effect of different neurons-per-core allocations.

Both networks are sFCNs, each with two hidden layers. The small network has dimensions $FC_1 = 256$ and $FC_2 = 128$, while the large network has dimensions $FC_1 = 1024$ and $FC_2 = 512$. Both networks, as shown in Fig 6, obtain similar accuracies on the testing set. The small network consists of 429 neurons and 38 544 connections, while the large network consists of 1581 neurons and 547 344 connections. The neurons in the small network, since they have fewer connections between each other, can be distributed more densely within the chip. Different configurations using 10, 20, 50, 100, 200, 500, and 1000 neurons-per-core were tested for the small network. For the large network, allocating less than 20 neurons-per-core results in using a second Loihi chip, thus a much slower and energy expensive processing, while configuring the network with more than 100 neurons-per-core is not possible due to hardware limitations regarding the maximum number of ingoing/outgoing connections from an individual core.

The small and large networks obtain similar latency and power consumption values when using the same neuron-per-core network allocation. The product between power

TABLE II
ARCHITECTURE OF THE *OPTIMAL* SNN MODEL. OPTIMAL WITH RESPECT TO: LATENCY L_{SAMPLE}, POWER P_{SAMPLE}, ENERGY E_{SAMPLE}, AND TOTAL NUMBER OF CORES OCCUPIED

Layer	Input	Output
Input Layer	-	$8 \times 1 \times 2$
Flattening Layer	$8 \times 1 \times 2$	1×16
FC_1 Layer	1×16	1×256
FC_2 Layer	1×256	1×128
Output Layer	1×128	1×13

and latency, that is, the energy, is higher for the large network $E_{\text{sample}} = 7814$ mJ compared to the small network $E_{\text{sample}} = 6612$ mJ. For the same neuron-per-core allocation, the small network occupies a fewer number of cores than the large network.

Since both networks obtain similar prediction accuracy, the configuration which achieves the optimal performance in terms of latency L_{sample}, power P_{sample}, energy E_{sample}, and total number of cores occupied is the *small* network and distributing 50 neurons per core. This results in an average latency $L_{\text{sample}} = 5.75$ ms, power $P_{\text{sample}} = 1150$ mW and $P_{\text{neurocore}} = 41$ mW, and energy $E_{\text{sample}} = 6$ ms $*$ 41 mW $=$ 246 mJ while using 12 cores. Table II illustrates the architecture of the final SNN.

C. Comparison of Performance Profile: GPU Versus Loihi

The run-time characterization for the SNN running on the Neuromorphic Chip is given in more detail in Section III-B. The same experimental procedure was repeated using the same sFCN running on a GPU, namely the NVIDIA GeForce RTX 2080 Super which operates with a clock frequency of 1650 MHz, has 3072 available CUDA cores, and can draw a maximum of 250 W.[2] The sample processing time L_{sample} for the SNN running on the GPU was obtained by inserting timers before and after the network *forward* function call, while the power consumption P_{sample} and GPU usage was obtained using the nvidia-smi tool.[3] The energy consumption E_{sample} was obtained by multiplying the power consumption by the processing time. The experiments were repeated five times and averaged. Running the SNN described in Table II results in a power consumption of $P^{\text{GPU}} = 52$ W using 1536 CUDA cores at a clock frequency of 1650 MHz.

Table III reports the average performance values for both the SNN running on the GPU and on the Neuromorphic Hardware. While the GPU has a slightly faster processing speed ($L^{\text{GPU}} = 1.4$ ms, $L^{\text{Loihi}} = 5.7$ ms), the SNN running on the Neuromorphic Hardware consumes a lot less power than its counterpart running on the GPU ($P^{\text{GPU}} = 52$ W, $P^{\text{Loihi}} = 41$ mW). This is consistent with findings of other works [17]. Furthermore, the SNN running on the GPU occupies $(1536/3072) = 50\%$ of available CUDA cores, while only $(12/256) = 4.7\%$ of neurocores on the Kapoho Bay.

D. Obtained Accuracies on NinaPro5 Testing Dataset

The reported accuracy values in Fig. 7 show that the proposed method outperforms common values found in the literature using the same dataset. The sFCN reaches an accuracy of 74% on the DB5 testing set, while other methods achieve accuracies below 70%.

The sFCN was trained using SLAYER according to the parameters in Section II-E for 1000 epochs and achieved a maximum training and testing accuracy of 77% and 74%. The obtained gesture-wise classification accuracies are summarized in Fig. 8 using a confusion matrix. While a training accuracy of 77% may seem poor, as mentioned in Section II-A, the dataset is highly unbalanced, does not contain many samples, and many gestures are highly similar. We argue that this

[2]https://www.nvidia.com/de-de/geforce/graphics-cards/rtx-2080-super/
[3]https://developer.nvidia.com/nvidia-system-management-interface

TABLE III
COMPARISON FOR THE SFCN RUNNING ON THE LOIHI AND THE GPU. BOTH * AND ** VALUES ARE OBTAINED FROM THE ENERGY PROBE OBJECTS DIRECTLY FROM INTEL'S API AND INDICATE THE ENERGY/POWER CONSUMED PER SAMPLE FOR *THE WHOLE SYSTEM** AND FOR THE NEUROCORES AND EMBEDDED* $\times$86 *ONLY***

Benchmark	GPU	Loihi
Latency	$1.4ms$	$5.7ms$
Energy	$320J$	$6.6J^*$, $246mJ^{**}$
Power	$52.0W$	$1.2W^*$, $41mW^{**}$
Usage	50%	4.7%

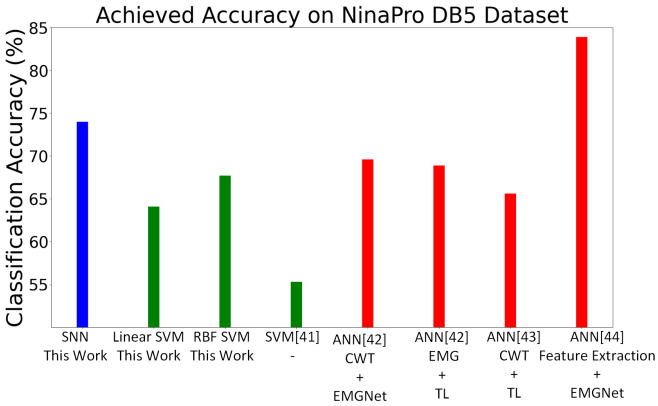


Fig. 7. Achieved accuracy in this article compared to other works using the same dataset. (TL: Transfer Learning.)

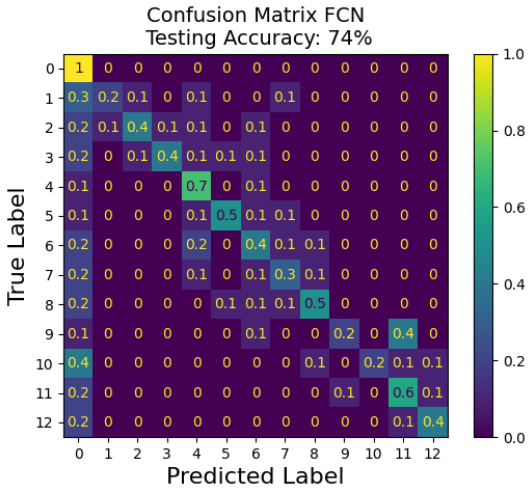


Fig. 8. Confusion matrix for the SNN running on the Loihi. Most wrong classifications occur due to the zero-label.

specific classification task can be considered as good. To show the benefit of our approach in a closed-loop system, we plan on acquiring a fully spiking sEMG dataset and re-running the gesture classification with our methodology.

IV. Conclusion

Existing methods rely on running conventional ML methods running on slow CPUs or power-hungry GPUs to solve the classification task. Our proposed method implements a light-weight and low-latency SNN running fully on a neuromorphic processor which takes the raw, spike-encoded EMG signal as input and returns the associated gesture.

For wearable devices, short processing latency is important to give a more realistic user interaction, while low power consumption is crucial to extend the autonomy of such devices. Our proposed EMG classification network running on the Loihi achieves an average latency of $L^{\text{Loihi}} = 5.7$ ms on input samples of 300 ms, achieves state-of-the-art accuracy of 74% on the popular NinaPro dataset, while consuming only $P^{\text{Loihi}} = 41$ mW. These benefits make neuromorphic edge devices, such as Intel's Kapoho Bay platform, ideal candidates for future wearable applications.

References

[1] L.-Y. Ma and N. Soin, "Recent progress in printed physical sensing electronics for wearable health-monitoring devices: A review," *IEEE Sensors J.*, vol. 22, no. 5, pp. 3844–3859, Mar. 2022.

[2] Z. Lv and Y. Li, "Wearable sensors for vital signs measurement: A survey," *J. Sensor Actuator Netw.*, vol. 11, no. 1, p. 19, Mar. 2022. [Online]. Available:https://www.mdpi.com/2224-2708/11/1/19

[3] I. C. Jeong, D. Bychkov, and P. C. Searson, "Wearable devices for precision medicine and health state monitoring," *IEEE Trans. Biomed. Eng.*, vol. 66, no. 5, pp. 1242–1258, May 2019.

[4] G. Aroganam, N. Manivannan, and D. Harrison, "Review on wearable technology sensors used in consumer sport applications," *Sensors*, vol. 19, no. 9, p. 1983, 2019.

[5] I. Awolusi, E. Marks, and M. Hallowell, "Wearable technology for personalized construction safety monitoring and trending: Review of applicable devices," *Autom. Construct.*, vol. 85, pp. 96–106, Jan. 2018.

[6] H. Kim, Y. Kwon, H. Lim, J. Kim, Y. Kim, and W. Yeo, "Recent advances in wearable sensors and integrated functional devices for virtual and augmented reality applications," *Adv. Funct. Mater.*, vol. 31, no. 39, Sep. 2021, Art. no. 2005692.

[7] M. Wu and J. Luo, "Wearable technology applications in healthcare: A literature review," *Online J. Nurs. Inform.*, vol. 23, no. 3, 2019.

[8] J. L. Betthauser *et al.*, "Stable responsive EMG sequence prediction and adaptive reinforcement with temporal convolutional networks," *IEEE Trans. Biomed. Eng.*, vol. 67, no. 6, pp. 1707–1717, Jun. 2020.

[9] G. Kumar, S. S. Yadav, Yogita, and V. Pal, "Machine learning-based framework to predict finger movement for prosthetic hand," *IEEE Sensors Lett.*, vol. 6, no. 6, pp. 1–4, Jun. 2022.

[10] R. R. Ilesan *et al.*, "Proof of concept in artificial-intelligence-based wearable gait monitoring for parkinson's disease management optimization," *Biosensors*, vol. 12, no. 4, p. 189, Mar. 2022. [Online]. Available: https://www.mdpi.com/2079-6374/12/4/189

[11] M. Rossi, M. Bianchi, E. Battaglia, M. G. Catalano, and A. Bicchi, "HapPro: A wearable haptic device for proprioceptive feedback," *IEEE Trans. Biomed. Eng.*, vol. 66, no. 1, pp. 138–149, Jan. 2019.

[12] X. Wang, M. Hersche, B. Tömekce, B. Kaya, M. Magno, and L. Benini, "An accurate EEGNet-based motor-imagery brain–computer interface for low-power edge computing," in *Proc. IEEE Int. Symp. Med. Meas. Appl. (MeMeA)*, Jun. 2020, pp. 1–6.

[13] M. Zanghieri, S. Benatti, A. Burrello, V. Kartsch, F. Conti, and L. Benini, "Robust real-time embedded EMG recognition framework using temporal convolutional networks on a multicore IoT processor," *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 2, pp. 244–256, Apr. 2020.

[14] P. Tsinganos, B. Cornelis, J. Cornelis, B. Jansen, and A. Skodras, "Improved gesture recognition based on sEMG signals and TCN," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1169–1173.

[15] S. Tam, M. Boukadoum, A. Campeau-Lecours, and B. Gosselin, "A fully embedded adaptive real-time hand gesture classifier leveraging HD-sEMG and deep learning," *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 2, pp. 232–243, Apr. 2020.

[16] A. Cisnal, J. Perez-Turiel, J.-C. Fraile, D. Sierra, and E. de la Fuente, "RobHand: A hand exoskeleton with real-time EMG-driven embedded Control. Quantifying hand gesture recognition delays for bilateral rehabilitation," *IEEE Access*, vol. 9, pp. 137809–137823, 2021.

[17] E. Ceolini *et al.*, "Hand-gesture recognition based on EMG and event-based camera sensor fusion: A benchmark in neuromorphic computing," *Frontiers Neurosci.*, vol. 14, p. 36, Aug. 2020.

[18] E. Covi *et al.*, "Adaptive extreme edge computing for wearable devices," *Frontiers Neurosci.*, vol. 15, p. 429, May 2021. [Online]. Available: https://www.frontiersin.org/article/10.3389/fnins.2021.611300

[19] S. Saadatnejad, M. Oveisi, and M. Hashemi, "LSTM-based ECG classification for continuous monitoring on personal wearable devices," *IEEE J. Biomed. Health Informat.*, vol. 24, no. 2, pp. 515–523, Feb. 2020, doi: 10.1109/JBHI.2019.2911367.

[20] D. V. Christensen *et al.*, "2022 roadmap on neuromorphic computing and engineering," 2021, *arXiv:2105.05956*.

[21] C. Mead, "How we created neuromorphic engineering," *Nature Electron.*, vol. 3, no. 7, pp. 434–435, 2020.

[22] A. Rubino, C. Livanelioglu, N. Qiao, M. Payvand, and G. Indiveri, "Ultra-low-power FDSOI neural circuits for extreme-edge neuromorphic intelligence," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 45–56, Jan. 2021.

[23] S. B. Furber *et al.*, "Overview of the SpiNNaker system architecture," *IEEE Trans. Comput.*, vol. 62, no. 12, pp. 2454–2467, Dec. 2013.

[24] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan. 2018.

[25] C. Mayr, S. Hoeppner, and S. Furber, "SpiNNaker 2: A 10 million core processor system for brain simulation and machine learning," 2019, *arXiv:1911.02385*.

[26] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, "A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (DYNAPs)," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 1, pp. 106–122, Feb. 2018.

[27] E. Chicca, F. Stefanini, C. Bartolozzi, and G. Indiveri, "Neuromorphic electronic circuits for building autonomous cognitive systems," *Proc. IEEE*, vol. 102, no. 9, pp. 1367–1388, Sep. 2014.

[28] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128×128 120 DB 15 us latency asynchronous temporal contrast vision sensor," *IEEE J. Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, Mar. 2008.

[29] C. Brandli, R. Berner, M. Yang, S.-C. Liu, and T. Delbruck, "A 240 × 180 130 DB 3 μs latency global shutter spatiotemporal vision sensor," *IEEE J. Solid-State Circuits*, vol. 49, no. 10, pp. 2333–2341, Sep. 2014.

[30] J. Anumula, D. Neil, T. Delbruck, and S.-C. Liu, "Feature representations for neuromorphic audio spike streams," *Frontiers Neurosci.*, vol. 12, p. 23, Feb. 2018. [Online]. Available: https://www.frontiersin.org/article/10.3389/fnins.2018.00023

[31] C. Bao, S. K. Seol, and W. S. Kim, "A 3D integrated neuromorphic chemical sensing system," *Sens. Actuators B, Chem.*, vol. 332, Apr. 2021, Art. no. 129527. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925400521000952

[32] N. Skatchkovsky, H. Jang, and O. Simeone, "Spiking neural networks—Part II: Detecting spatio-temporal patterns," *IEEE Commun. Lett.*, vol. 25, no. 6, pp. 1741–1745, Jun. 2021.

[33] X. Wang, X. Lin, and X. Dang, "Supervised learning in spiking neural networks: A review of algorithms and evaluations," *Neural Netw.*, vol. 125, pp. 258–280, May 2020.

[34] R. Ambati, S. Raja, M. Al-Hameed, T. John, Y. Arjoune, and R. Shekhar, "Neuromorphic architecture accelerated automated seizure detection in multi-channel scalp EEG," *Sensors*, vol. 22, no. 5, p. 1852, Feb. 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/5/1852

[35] E. Donati, M. Payvand, N. Risi, R. Krause, and G. Indiveri, "Discrimination of EMG signals using a neuromorphic implementation of a spiking neural network," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 5, pp. 795–803, Oct. 2019.

[36] F. Corradi *et al.*, "ECG-based heartbeat classification in neuromorphic hardware," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2019, pp. 1–8.

[37] F. C. Bauer, D. R. Muir, and G. Indiveri, "Real-time ultra-low power ECG anomaly detection using an event-driven neuromorphic processor," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 6, pp. 1575–1582, Dec. 2019.

[38] M. Sharifshazileh, K. Burelo, J. Sarnthein, and G. Indiveri, "An electronic neuromorphic system for real-time detection of high frequency oscillations (HFO) in intracranial EEG," *Nature Commun.*, vol. 12, no. 1, pp. 1–14, Dec. 2021.

[39] P. Zhang, J. Zhang, and A. Elsabbagh, "Lower limb motion intention recognition based on sEMG fusion features," *IEEE Sensors J.*, vol. 22, no. 7, pp. 7005–7014, Apr. 2022.

[40] T. Stefanou, D. Guiraud, C. Fattal, C. Azevedo-Coste, and L. Fonseca, "Frequency-domain sEMG classification using a single sensor," *Sensors*, vol. 22, no. 5, p. 1939, Mar. 2022. [Online]. Available:https://www.mdpi.com/1424-8220/22/5/1939

[41] S. Pizzolato, L. Tagliapietra, M. Cognolato, M. Reggiani, H. Müller, and M. Atzori, "Comparison of six electromyography acquisition setups on hand movement classification tasks," *PLoS ONE*, vol. 12, no. 10, Oct. 2017, Art. no. e0186132.

[42] L. Chen, J. Fu, Y. Wu, H. Li, and B. Zheng, "Hand gesture recognition using compact CNN via surface electromyography signals," *Sensors*, vol. 20, no. 3, p. 672, 2020.

[43] U. Cătăllard *et al.*, "Deep learning for electromyographic hand gesture signal classification using transfer learning," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 27, no. 4, pp. 760–771, Oct. 2019.

[44] W. Wei, Q. Dai, Y. Wong, Y. Hu, M. Kankanhalli, and W. Geng, "Surface-electromyography-based gesture recognition by multi-view deep learning," *IEEE Trans. Biomed. Eng.*, vol. 66, no. 10, pp. 2964–2973, Oct. 2019.

[45] Zhang, Yang, Qian, and Zhang, "Real-time surface EMG pattern recognition for hand gestures based on an artificial neural network," *Sensors*, vol. 19, no. 14, p. 3170, Jul. 2019. [Online]. Available:https://pubmed.ncbi.nlm.nih.gov/31323888

[46] J. Behrenbeck *et al.*, "Classification and regression of spatio-temporal signals using NeuCube and its realization on SpiNNaker neuromorphic hardware," *J. Neural Eng.*, vol. 16, no. 2, Apr. 2019, Art. no. 026014.

[47] M. Atzori *et al.*, "Building the ninapro database: A resource for the biorobotics community," in *Proc. 4th IEEE RAS EMBS Int. Conf. Biomed. Robot. Biomechtron. (BioRob)*, Jun. 2012, pp. 1258–1265.

[48] S. Bam Shrestha and G. Orchard, "SLAYER: Spike layer error reassignment in time," 2018, *arXiv:1810.08646*.

[49] S. Pancholi, A. M. Joshi, and D. Joshi, "A robust and accurate deep learning based pattern recognition framework for upper limb prosthesis using sEMG," 2021, *arXiv:2106.02463*.

[50] A. Jaramillo-Yánez, M. E. Benalcázar, and E. Mena-Maldonado, "Real-time hand gesture recognition using surface electromyography and machine learning: A systematic literature review," *Sensors*, vol. 20, no. 9, pp. 1–36, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/9/2467

[51] M. Atzori *et al.*, "Electromyography data for non-invasive naturally-controlled robotic hand prostheses," *Sci. Data*, vol. 1, Dec. 2014, Art. no. 140053, doi: 10.1038/sdata.2014.53.

[52] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: Opportunities and challenges," *Frontiers Neurosci.*, vol. 12, p. 774, Oct. 2018. [Online]. Available: https://www.frontiersin.org/article/10.3389/fnins.2018.00774

[53] E. Donati *et al.*, "Processing EMG signals using reservoir computing on an event-based neuromorphic system," in *Proc. IEEE Biomed. Circuits Syst. Conf. (BioCAS)*, Oct. 2018, pp. 1–4.

[54] F. Corradi and G. Indiveri, "A neuromorphic event-based neural recording system for smart brain-machine-interfaces," *IEEE Trans. Biomed. Circuits Syst.*, vol. 9, no. 5, pp. 699–709, Oct. 2015, doi: 10.5167/uzh-121693.

[55] C. J. C. Burges, "A tutorial on support vector machines for pattern recognition," *Data Mining Knowl. Discovery*, vol. 2, no. 2, pp. 121–167, 1998, doi: 10.1023/A:1009715923555.

[56] M. Davies, "Advancing neuromorphic computing with Loihi: A survey of results and outlook," *Proc. IEEE*, vol. 109, no. 5, pp. 911–934, May 2021.

[57] D. Rasmussen, "NengoDL: Combining deep learning and neuromorphic modelling methods," *Neuroinformatics*, vol. 17, no. 4, pp. 611–628, Oct. 2019.

[58] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, "Conversion of continuous-valued deep networks to efficient event-driven networks for image classification," *Frontiers Neurosci.*, vol. 11, p. 682, Dec. 2017. [Online]. Available: https://www.frontiersin.org/article/10.3389/fnins.2017.00682

[59] S. M. Bohte, J. N. Kok, and H. L. Poutre, "Error-backpropagation in temporally encoded networks of spiking neurons," *Neurocomputing*, vol. 48, nos. 1–4, pp. 17–37, 2002.

[60] S. B. Shrestha and Q. Song, "Robust spike-train learning in spike-event based weight update," *Neural Netw.*, vol. 96, pp. 33–46, Dec. 2017.